J
3

Session 1 · Jour 3 / 9

Kafka Streams Kafka Connect & ksqlDB

FORMATION APACHE KAFKA · 9 JOURS

Apache Kafka de A à Z

Kafka Streams Connect & ksqlDB

Kafka Streams

KStream · KTable · Fenêtres · Jointures

Agrégations temps réel

groupBy · count · reduce · aggregate

Kafka Connect

Source/Sink connectors · JDBC · SMT

ksqlDB

SQL sur des streams · Tables matérialisées

⏱ 1 jour · 40% théorie / 60% TP · 50 slides | Prérequis : Jour 1+2 validés · Java 17 · Maven · Docker

Plan du Jour 3 — Programme complet (50 slides)

01 — 9h00–10h30
Kafka Streams — KStream, KTable & Transformations — Slides 3–12

02 — 10h45–12h00
Kafka Streams — Agrégations, Fenêtres & Jointures — Slides 13–22

03 — 13h00–14h30
Kafka Connect — Architecture & Connecteurs — Slides 23–32

04 — 14h30–15h30
ksqlDB — SQL sur des Streams — Slides 33–40

05 — 15h45–17h30
TP — Pipeline temps réel bout en bout — Slides 41–50

Pause café 10h30–10h45 · Déjeuner 12h00–13h00 · Pause 15h30–15h45 · Fin 17h30

01

Kafka Streams
KStream, KTable & Transformations

Stream processing natif · Topologie de traitement · Transformations sans cluster

Kafka Streams — Bibliothèque de stream processing

Dimension	KStream — Flux d'événements	KTable — Table d'état courant
Sémantique	Append-only : chaque message est un événement indépendant	Upsert : le dernier message par clé remplace le précédent
Exemple	order-1 CREATED → order-1 PAID → order-1 SHIPPED → 3 événements distincts tous traités	order-1 → dernier statut (SHIPPED) Les états précédents sont oubliés
API Java	**builder.stream("commerce.orders")**	**builder.table("commerce.orders")**
Cas d'usage	Logs · Events · Transactions · Audit CDC · Chaque event compte	État courant · Stock · Profils · Prix · Inventaire temps réel

KStream vs KTable — Deux abstractions fondamentales

KStream — Flux d'événements

Représente un flux infini et immuable d'enregistrements. Chaque enregistrement est un événement indépendant. Append-only : un nouvel événement n'efface pas les précédents. Chaque message est traité individuellement.

```
// Chaque commande est un événement distinct
// offset=0: {"id":1, "status":"CREATED", "amount":99}
// offset=1: {"id":2, "status":"CREATED", "amount":49}
// offset=2: {"id":1, "status":"PAID",    "amount":99}
// offset=3: {"id":1, "status":"SHIPPED", "amount":99}
// → 4 événements, order-1 apparaît 3 fois !
KStream<String, Order> ordersStream =
    builder.stream("commerce.orders");
```

📌 Cas d'usage : logs, events, transactions, audit, CDC — chaque événement compte

KTable — Table de faits (changelog)

Représente l'état COURANT de chaque clé. Chaque nouveau message pour une clé écrase le précédent. Sémantique upsert : photo instantanée de l'état actuel. Matérialisé en RocksDB localement.

```
// La KTable ne garde que le DERNIER statut par ID
// order-1 → {"status":"SHIPPED", "amount":99.99}
//           (les events CREATED et PAID sont oubliés)
// order-2 → {"status":"CREATED", "amount":49.00}
// order-3 → {"status":"PAID",    "amount":199.00}
// → 3 entrées = 3 commandes distinctes (état actuel)
KTable<String, Order> ordersTable =
    builder.table("commerce.orders");
```

📌 Cas d'usage : état courant, profils utilisateurs, stock en temps réel, prix

Règle : KStream pour chaque événement · KTable pour l'état courant · GlobalKTable pour les données de référence (broadcast).

Topologie Kafka Streams — Processors, Sources & Sinks

🗺 Topologie de traitement

Une application Kafka Streams définit une topologie = graphe acyclique dirigé (DAG) de processors. Chaque processor reçoit des enregistrements, les transforme et les transmet aux suivants.

Topologie simple — Filtrer et transformer

```
StreamsBuilder builder = new StreamsBuilder();

// ─── SOURCE ──────────────────────────────────────────────────
KStream<String, String> raw =
    builder.stream("commerce.orders"); // Source topic

// ─── PROCESSORS ──────────────────────────────────────────────
KStream<String, Order> orders = raw
    // 1. Désérialiser en objets Order
    .mapValues(json -> Order.fromJson(json))
    // 2. Filtrer : garder uniquement CREATED
    .filter((key, order) ->
        order.getStatus() == OrderStatus.CREATED)
    // 3. Enrichir avec un timestamp de traitement
    .mapValues(order -> {
        order.setProcessedAt(Instant.now());
        return order;
    })
    // 4. Changer la clé = région de la commande
    .selectKey((key, order) -> order.getRegion());

// ─── SINK ────────────────────────────────────────────────────
orders.to("commerce.orders.new"); // Sink topic
```

Types de processors

Source Processor

Lit depuis un topic Kafka · Convertit en KStream/KTable

Stream Processor

Transforme les données · map · filter · flatMap · branch

Sink Processor

Écrit dans un topic Kafka · to() ou through()

State Store

RocksDB local · Utilisé par aggregate/join · Changelog topic

Transformations Stateless — map, filter, flatMap, branch

⚡ Opérations Stateless

Les opérations stateless ne maintiennent pas d'état entre les enregistrements. Elles traitent chaque message indépendamment — très performantes et facilement parallélisables.

Transformations stateless essentielles

```
KStream<String, Order> orders = builder.stream("orders");

// ─── FILTER : garder seulement les commandes > 100€
KStream<String, Order> highValue =
    orders.filter((key, order) -> order.getAmount() > 100.0);

// ─── MAP : transformer clé + valeur
KStream<String, String> summaries = orders
    .map((key, order) -> KeyValue.pair(
        order.getCustomerId(),  // Nouvelle clé
        order.getStatus().name())); // Nouvelle valeur

// ─── MAP VALUES : transformer valeur seulement (plus rapide)
KStream<String, OrderDTO> dtos =
    orders.mapValues(order -> new OrderDTO(order));

// ─── FLAT MAP : 1 message → N messages
KStream<String, String> items = orders
    .flatMapValues(order -> order.getItems()
        .stream()
        .map(item -> item.getSku())
        .collect(Collectors.toList()));

// ─── BRANCH : router vers plusieurs streams
```

Référence des opérations

`filter(predicate)`	Garde si predicate=true · Stateless
`map(mapper)`	Change clé ET valeur · Peut changer le type
`mapValues(mapper)`	Change valeur seulement · Plus rapide (pas de re-partition)
`flatMapValues(mapper)`	1 valeur → N valeurs · Ex: items d'une commande
`flatMap(mapper)`	1 KeyValue → N KeyValues · Re-partition possible
`peek(action)`	Observe sans modifier · Debug · Métriques
`selectKey(selector)`	Changer la clé → déclenche re-partition !
`branch() / split()`	Router vers plusieurs streams selon des prédicats

selectKey() et map() peuvent déclencher une re-partition si la clé change — opération coûteuse, utiliser mapValues() quand possible.

Configuration Kafka Streams — Paramètres essentiels

Configuration complète application.id

```
Properties props = new Properties();

// ─── IDENTITÉ ──────────────────────────────────────────────────
// application.id = Consumer Group ID des streams
// UNIQUE par application, stable entre redémarrages
props.put(StreamsConfig.APPLICATION_ID_CONFIG,
    "order-processor-v1");
props.put(StreamsConfig.BOOTSTRAP_SERVERS_CONFIG,
    "broker1:9092,broker2:9092");

// ─── SÉRIALISATION DÉFAUT ──────────────────────────────────────
props.put(StreamsConfig.DEFAULT_KEY_SERDE_CLASS_CONFIG,
    Serdes.String().getClass());
props.put(StreamsConfig.DEFAULT_VALUE_SERDE_CLASS_CONFIG,
    Serdes.String().getClass());

// ─── PERFORMANCE ───────────────────────────────────────────────
props.put(StreamsConfig.NUM_STREAM_THREADS_CONFIG, 4);
props.put(StreamsConfig.COMMIT_INTERVAL_MS_CONFIG, 1000);
```

pom.xml — Dépendance Maven

```
<dependency>
  <groupId>org.apache.kafka</groupId>
  <artifactId>kafka-streams</artifactId>
  <version>3.6.1</version>
</dependency>
```

Paramètres clés

`application.id`
ID unique de l'app = Consumer Group

`num.stream.threads`
Nb threads (= partitions / nœud)

`commit.interval.ms`
Fréquence de commit état (1s)

`cache.max.bytes`
Cache RocksDB (10MB défaut)

`processing.guarantee`
EXACTLY_ONCE_V2 en prod

`replication.factor`
RF des changelog topics (=3)

`state.dir`
/tmp/kafka-streams (ou volume)

✔ Scalabilité : lancer N instances avec le même application.id → partitions automatiquement réparties

Serdes Kafka Streams — Sérialisation des données

📦 Serdes (Serializer + Deserializer)

Un Serde est la combinaison d'un Serializer et d'un Deserializer. Kafka Streams utilise des Serdes pour (dé)sérialiser clés et valeurs dans la topologie. Kafka fournit des Serdes natifs pour les types communs.

Serdes natifs Kafka

```
// Serdes intégrés
Serdes.String()   // StringSerializer/Deserializer
Serdes.Integer()  // IntegerSerializer/Deserializer
Serdes.Long()     // LongSerializer/Deserializer
Serdes.Double()   // DoubleSerializer/Deserializer
Serdes.Bytes()    // BytesSerializer/Deserializer
Serdes.ByteArray()// byte[] raw

// Usage dans stream
KStream<String, Long> counts =
    orders.mapValues(order -> order.getAmount().longValue());
// Serde explicite si différent du défaut :
counts.to("order.totals",
    Produced.with(Serdes.String(), Serdes.Long()));
```

Serde Avro avec Schema Registry

```
// Dépendances Maven :
// io.confluent:kafka-streams-avro-serde:7.6.0

// Usage Avro Serde
SpecificAvroSerde<Order> orderSerde = new SpecificAvroSerde<>();
Map<String, String> serdeConfig = Map.of(
  "schema.registry.url", "http://sr:8081");
orderSerde.configure(serdeConfig, false);

// Dans la topologie
KStream<String, Order> orders =
    builder.stream("commerce.orders",
        Consumed.with(Serdes.String(), orderSerde));
```

orders

Serde JSON avec Jackson

```
// Serde JSON générique
public class JsonSerde<T> implements Serde<T> {
    private final ObjectMapper mapper = new ObjectMapper();
    private final Class<T> targetType;
    public JsonSerde(Class<T> targetType) {
this.targetType=targetType; }
```

Serde dans le StreamsBuilder

```
// Serde défaut global dans les props :
props.put(DEFAULT_KEY_SERDE, Serdes.String().getClass());
props.put(DEFAULT_VALUE_SERDE, Serdes.String().getClass());
// Override local avec Consumed.with() / Produced.with()
```

Bonne pratique : définir les Serdes par défaut dans les props et les surcharger localement si nécessaire.

Scalabilité & Fault Tolerance — Kafka Streams en production

⚡ Scalabilité Kafka Streams

Kafka Streams scale horizontalement : lancer N instances avec le même application.id. Chaque instance reçoit automatiquement un sous-ensemble de partitions. Ajouter une instance = rebalancing automatique.

Instance 1 (node-1)	Instance 2 (node-2)	Instance 3 (node-3)
P0+P1 de commerce.orders · State store local RocksDB · 2 partitions traitées	P2+P3 de commerce.orders · State store local RocksDB · 2 partitions traitées	P4+P5 de commerce.orders · State store local RocksDB · 2 partitions traitées

Fault Tolerance — RocksDB changelog

L'état local RocksDB est répliqué automatiquement dans un topic Kafka interne (changelog topic). En cas de panne d'une instance, l'état est restauré depuis le changelog → reprise rapide.

Standby Replicas — num.standby.replicas

Paramètre qui crée des copies chaudes de l'état sur d'autres instances. En cas de panne, une instance standby prend le relais immédiatement sans attendre la restauration du changelog.

Monitoring Streams — StreamsMetrics

Métriques JMX natives : process-rate, commit-rate, lag (distance entre offset de lecture et LEO du topic source). Kafka Streams expose des métriques détaillées par thread et par task.

num.standby.replicas=1 en prod → failover instantané sans restauration. Coût : 2× plus de partitions standby.

Interactive Queries — Lire l'état depuis l'extérieur

🔍 Interactive Queries

Kafka Streams permet d'exposer l'état local RocksDB via des requêtes depuis l'extérieur. Chaque instance peut répondre pour ses propres partitions. Pour les autres partitions : proxy vers l'instance correcte.

Interactive Queries — REST API sur l'état

```
// Dans la topologie : créer un state store nommé
KTable<String, Long> orderCounts = orders
    .groupByKey()
    .count(Materialized.as("order-counts-store"));
// Store RocksDB local exposable en read-only

// Exposer via REST (Spring Boot)
@RestController
public class OrderQueryController {

    @Autowired
    private KafkaStreams streams;

    @GetMapping("/orders/{customerId}/count")
    public Long getOrderCount(@PathVariable String customerId) {
        // Accéder au state store local
        ReadOnlyKeyValueStore<String, Long> store =
            streams.store(StoreQueryParameters
                .fromNameAndType(
                    "order-counts-store",
                    QueryableStoreTypes.keyValueStore()));

        Long count = store.get(customerId);
        return count != null ? count : 0L;
```

Store types disponibles

KeyValueStore

get(key), range(from,to), all()

WindowStore

fetch(key, from, to) par fenêtre temporelle

SessionStore

fetch(key) par session utilisateur

ReadOnly variants

Sécurité : accès lecture seule

Multi-instances : utiliser HostInfo pour proxier vers la bonne instance

Quiz — Section 01 : Kafka Streams Fondamentaux

❓ KStream vs KTable : quelle est la différence principale ?

- ○ KStream est plus rapide que KTable
- ✔ **KStream = flux d'événements (chaque msg compte) · KTable = état courant par clé (upsert)**
- ○ KTable ne supporte pas le filtrage
- ○ KStream est utilisé pour les jointures

❓ Comment scaler une application Kafka Streams ?

- ○ Ajouter des partitions au topic source
- ✔ **Lancer N instances avec le même application.id — partitions réparties automatiquement**
- ○ Augmenter num.stream.threads seulement
- ○ Ajouter des brokers Kafka

❓ Que fait selectKey() dans une topologie ?

- ○ Filtre par clé
- ✔ **Change la clé du message → peut déclencher une re-partition**
- ○ Agrège par clé
- ○ Supprime la clé

❓ Où est stocké l'état RocksDB en production ?

- ○ Dans un topic Kafka standard
- ✔ **Localement sur l'instance + répliqué dans un changelog topic**
- ○ Dans ZooKeeper
- ○ En mémoire uniquement

02

Agrégations, Fenêtres & Jointures

groupBy · count · aggregate · Tumbling · Hopping · Session · Stream-Table Join

Agrégations Kafka Streams — groupBy, count, aggregate

📊 Agrégations Stateful

Les agrégations maintiennent un état local (RocksDB) mis à jour à chaque nouveau message. Elles produisent une KTable en sortie dont les valeurs sont mises à jour en continu.

Agrégations — count, reduce, aggregate

```
KStream<String, Order> orders =
    builder.stream("commerce.orders",
        Consumed.with(Serdes.String(), orderSerde));

// ─── COUNT : compter par client ──────────────────────────────
KTable<String, Long> orderCountByCustomer = orders
    .groupBy((key, order) ->
        KeyValue.pair(order.getCustomerId(), order))
    .count(Materialized.as("order-count-store"));

// ─── REDUCE : somme des montants par client ──────────────────
KTable<String, Double> totalByCustomer = orders
    .groupBy((key, order) ->
        KeyValue.pair(order.getCustomerId(), order))
    .aggregate(
        () -> 0.0,  // Initializer
        (custId, order, total) ->
            total + order.getAmount(),  // Aggregator (add)
        (custId, order, total) ->
            total - order.getAmount(),  // Subtractor (undo)
        Materialized.<String, Double, KeyValueStore<...>>as(
            "total-amount-store")
            .withValueSerde(Serdes.Double())
    );
```

Méthodes d'agrégation

`groupByKey()`
Grouper par la clé existante · Pas de re-partition

`groupBy(selector)`
Grouper par nouvelle clé · Re-partition !

`count()`
Compter les messages par clé · → KTable<K, Long>

`reduce(reducer)`
Combiner avec opération commutative

`aggregate(init, agg, sub)`
Agrégation générique avec type différent

`toStream()`
Convertir KTable → KStream pour écriture topic

groupBy() déclenche une re-partition (nouveau topic interne) — utiliser groupByKey() si la clé est déjà correcte.

Fenêtres Temporelles — Tumbling, Hopping, Session

⏱️ Windowing — Traitement par fenêtres de temps

Les fenêtres temporelles permettent de grouper des événements dans des intervalles de temps et de calculer des agrégats sur ces intervalles. Chaque fenêtre produit un résultat indépendant.

Tumbling Window

Fenêtres disjointes fixes

```
// CA par heure (fenêtres 1h
non-chevauchantes)
orders
  .groupByKey()
  .windowedBy(TimeWindows

.ofSizeWithNoGrace(Duration.ofHours(1))
)
  .aggregate(
    () -> 0.0,
    (k, v, total) -> total +
v.getAmount(),
    Materialized.with(Serdes.String(),
Serdes.Double()))
  .toStream()
  .map((wk, v) -> KeyValue.pair(
    wk.window().startTime().toString(),
v))
  .to("analytics.hourly-revenue");
// Résultat : une valeur par heure par
clé
```

09:00–10:00 | 10:00–11:00 | 11:00–12:00
Fenêtres non-chevauchantes · Idéal : rapports horaires

Hopping Window

Fenêtres chevauchantes

```
// Moyenne mobile 10min, saut de 5min
orders
  .groupByKey()

.windowedBy(TimeWindows.ofSizeAndGrace(
    Duration.ofMinutes(10),  // taille
fenêtre
    Duration.ofSeconds(30))) // grace
period

.count(Materialized.as("hopping-count")
)
  .toStream()
  .to("analytics.rolling-counts");
// 09:00–09:10 · 09:05–09:15 ·
09:10–09:20
// Chevauchement = données dans
plusieurs fenêtres
```

09:00–09:10 | 09:05–09:15 | 09:10–09:20
Chevauchement · Idéal : alertes, détection anomalies

Session Window

Fenêtres d'inactivité

```
// Sessions utilisateur (inactif après
30min)
clickstream
  .groupByKey()
  .windowedBy(SessionWindows
    .ofInactivityGapWithNoGrace(
      Duration.ofMinutes(30)))

.count(Materialized.as("session-count")
)
  .toStream()
  .map((wk, v) -> KeyValue.pair(
    wk.key() + "@" +
    wk.window().startTime(), v))
  .to("analytics.user-sessions");
// Durée variable par session
utilisateur
```

Début activité → fin après 30min inactivité
Idéal : analytics comportementaux, UX

Grace period : temps supplémentaire pour accepter les messages en retard (late arrivals). ofSizeAndGrace() en prod.

Jointures — Stream-Stream, Stream-Table, Table-Table

🔗 Jointures dans Kafka Streams

Kafka Streams supporte 3 types de jointures. Les données doivent être co-partitionnées (même nombre de partitions, même clé) pour les stream-stream et stream-table joins.

Les 3 types de jointures

```
// ─── STREAM-STREAM JOIN ──────────────────────────────────────

// Rejoindre 2 streams dans une fenêtre de temps
KStream<String, Order> orders =
    builder.stream("commerce.orders");
KStream<String, Payment> payments =
    builder.stream("commerce.payments");

KStream<String, EnrichedOrder> enriched = orders.join(
    payments,
    (order, payment) -> new EnrichedOrder(order, payment),
    JoinWindows.ofTimeDifferenceAndGrace(
        Duration.ofMinutes(5),  // fenêtre de jointure
        Duration.ofSeconds(30)),
    StreamJoined.with(Serdes.String(),
        orderSerde, paymentSerde));

// ─── STREAM-TABLE JOIN ───────────────────────────────────────

// Enrichir un stream avec une table de référence
KTable<String, Customer> customers =
    builder.table("commerce.customers");
KStream<String, EnrichedOrder> withCustomer = orders.join(
    customers
```

Types de jointures

`join()`
Inner Join · Les deux côtés doivent exister

`leftJoin()`
Left Join · Résultat même si droite absente

`outerJoin()`
Full Outer Join · Stream-Stream uniquement

Co-partitionnement

- ⚠️ Même nombre de partitions sur les deux topics
- ⚠️ Même sérialisation de la clé (même type)
- ⚠️ Même partitionnement logique (même hash function)
- ⚠️ Utiliser GlobalKTable pour éviter le co-partitionnement

GlobalKTable diffuse les données sur toutes les partitions — idéal pour les tables de référence (clients, produits).

GlobalKTable & Re-partitionnement

KTable standard

```
// Co-partitionné — jointure locale
// Chaque instance ne voit que ses partitions
KTable<String, Customer> customers =
    builder.table("commerce.customers",
        Consumed.with(Serdes.String(), customerSerde));

// Jointure possible uniquement si
// orders et customers ont le même nb partitions
// et la même clé (customerId)
orders.join(customers, (order, cust) ->
    order.withCustomer(cust));
// ⚠️ Ne fonctionne pas si co-partitionnement absent !
```

Avantages : mise à jour locale · Performances · Changelog topic
Inconvénients : co-partitionnement obligatoire

GlobalKTable — Broadcast complet

```
// Toutes les données diffusées à TOUTES les instances
// Chaque instance a une copie COMPLÈTE de la table
KGlobalTable<String, Product> products =
    builder.globalTable("catalogue.products",
        Consumed.with(Serdes.String(), productSerde));

// Jointure sans co-partitionnement !
// Clé de jointure peut être différente de la clé du topic
orders.join(
    products,
    (orderKey, order) -> order.getProductSku(), // clé de rech.
    (order, product) -> order.withProduct(product));
// Pas de re-partition · Disponible sur toutes les instances
```

Avantages : pas de co-partitionnement · Jointure par clé différente
Inconvénients : volume limité (toute la table en mémoire/RocksDB)

Règle : GlobalKTable pour les tables de référence < 1 Go · KTable standard pour les grandes tables avec co-partitionnement.

Exactly-Once dans Kafka Streams

🎯 **EXACTLY_ONCE_V2 — Traitement garanti**

Kafka Streams peut garantir que chaque enregistrement est traité exactement une fois end-to-end, y compris les agrégations et les jointures. Activé via processing.guarantee=EXACTLY_ONCE_V2 (Kafka 2.6+).

AT_LEAST_ONCE (défaut)

`processing.guarantee=at_least_once`

- ✔ Meilleure performance
- ✔ Messages rejoués en cas de panne
- ⚠️ Doublons possibles dans les agrégats
- ⚠️ Changer la clé → pas de re-partition atomique

📌 Cas : logs non-critiques, analytics approximatives

EXACTLY_ONCE_V2 (prod)

`processing.guarantee=exactly_once_v2`

- ✔ Aucun doublon garanti
- ✔ Atomique : lire + traiter + écrire
- ⚠️ Changelog + sink écrits en transaction
- ⚠️ Performance légèrement réduite (~20%)

📌 Cas : finance, facturation, CDC critique

EXACTLY_ONCE_V2 (Kafka 2.6+) utilise les transactions côté consumer pour le commit d'offset + écriture atomique.

Testing — TopologyTestDriver sans Kafka

🧪 TopologyTestDriver

Kafka Streams fournit un TopologyTestDriver pour tester les topologies sans cluster Kafka. Tests unitaires rapides, déterministes, sans I/O réseau. Inclus dans kafka-streams-test-utils.

Test complet avec TopologyTestDriver

```
@Test
public void testOrderFiltering() {
    // ─── Setup ──────────────────────────────────────────
    StreamsBuilder builder = new StreamsBuilder();
    OrderProcessor processor = new OrderProcessor();
    processor.buildTopology(builder); // ta topologie
    Topology topology = builder.build();

    Properties props = new Properties();
    props.put(APPLICATION_ID_CONFIG, "test-app");
    props.put(BOOTSTRAP_SERVERS_CONFIG, "dummy:9092");

    try (TopologyTestDriver driver =
            new TopologyTestDriver(topology, props)) {

        // Input topic
        TestInputTopic<String, String> inputTopic =
            driver.createInputTopic("commerce.orders",
                new StringSerializer(), new StringSerializer());

        // Output topic
        TestOutputTopic<String, String> outputTopic =
            driver.createOutputTopic("commerce.orders.new",
                new StringDeserializer()
```

Test des State Stores

Test agrégation

```
// Vérifier l'état du store
inputTopic.pipeInput("cust-1",
    orderJson(99.99));
inputTopic.pipeInput("cust-1",
    orderJson(49.99));

// Lire le store
KeyValueStore<String, Double> store =
    driver.getKeyValueStore(
        "total-amount-store");

Double total = store.get("cust-1");
assertEquals(149.98, total, 0.001);
```

✔ Avantages
- Tests sans cluster Kafka
- Rapides (ms)
- Déterministes
- Intégration JUnit/Mockito

Monitoring Kafka Streams — Métriques et health checks

📊 Métriques Kafka Streams

Kafka Streams expose des métriques JMX détaillées par thread, task et store. Intégration native avec Micrometer (Spring Boot) et Prometheus pour le monitoring en production.

Health check et métriques Spring Boot

```
@Component
public class StreamsHealthCheck implements HealthIndicator {

    @Autowired
    private KafkaStreams kafkaStreams;

    @Override
    public Health health() {
        KafkaStreams.State state = kafkaStreams.state();
        if (state == KafkaStreams.State.RUNNING) {
            return Health.up()
                .withDetail("state", state.name())
                .withDetail("threads",
                    kafkaStreams.metadataForAllPartitionsAndHosts()
                        .size())
```

États Kafka Streams : CREATED → REBALANCING → RUNNING → PENDING_SHUTDOWN → NOT_RUNNING
Alerter si état != RUNNING pendant > 30 secondes

Métriques clés JMX

`process-rate`	Msgs traités / sec par thread
`commit-rate`	Commits / sec (état)
`poll-records-avg`	Msgs par poll call
`put-rate (store)`	Écritures RocksDB/sec
`suppression-emit-rate`	Sorties de suppressions fenêtrées
`task-created-rate`	Rebalancing : tasks créées

Intégration Micrometer : KafkaStreamsMetricsAutoConfiguration (Spring Boot) → metrics auto vers Prometheus.

Patterns avancés — Punctuator, repartition, through()

Punctuator — Traitement planifié

```
// Exécuter une action périodique
// Ex : flush vers une base de données toutes les 30s
builder.stream("orders")
    .process(() -> new Processor<String, Order>() {
        private ProcessorContext context;
        private Map<String, Order> buffer = new HashMap<>();

        @Override
        public void init(ProcessorContext ctx) {
            this.context = ctx;
            // Schedule : toutes les 30 secondes
            ctx.schedule(Duration.ofSeconds(30),
                PunctuationType.WALL_CLOCK_TIME,
                timestamp -> {
                    // Flush buffer vers DB
```

repartition() — Re-partitionnement explicite

```
// repartition() est la version moderne de through()
KStream<String, Order> byRegion = orders
    .selectKey((k, v) -> v.getRegion())
    .repartition(Repartitioned
        .with(Serdes.String(), orderSerde)
        .withNumberOfPartitions(12)
        .withName("orders-by-region"));

// Plus sûr : nom explicite du topic
// Contrôle sur le nb de partitions
// Kafka Streams gère le cycle de vie
```

through() — Écrire et relire

```
// Écrire dans un topic intermédiaire puis relire
// Utile pour forcer une re-partition
KStream<String, Order> repartitioned = orders
    .selectKey((k, v) -> v.getRegion())
    .through("orders-by-region",
        Produced.with(Serdes.String(), orderSerde));
// → Nouveau topic créé + relu depuis le début
```

foreach() vs peek()

```
// peek() : observer sans modifier (debug)
orders
    .peek((key, value) ->
        log.debug("Processing: {} → {}", key, value))
    .filter(...)

// foreach() : terminal, pas de retour KStream
orders.foreach((key, value) -> {
    externalService.call(value);
    // ⚠️ Terminal : aucune autre opération après
});
```

repartition() préférable à through() depuis Kafka 2.6 — gestion automatique du topic interne.

Quiz — Section 02 : Agrégations, Fenêtres & Jointures

❓ Quelle est la différence entre Tumbling et Hopping window ?

- ○ Tumbling = plus grande
- ✔ **Tumbling = fenêtres disjointes (pas de chevauchement) · Hopping = fenêtres qui se chevauchent**
- ○ Hopping est plus rapide
- ○ Pas de différence

❓ Quand utiliser GlobalKTable vs KTable ?

- ○ Toujours GlobalKTable, c'est plus simple
- ✔ **GlobalKTable pour les petites tables de référence (pas de co-partitionnement requis) · KTable pour les grandes tables**
- ○ KTable est obsolète
- ○ Aucune différence

❓ TopologyTestDriver permet de :

- ○ Visualiser graphiquement la topologie
- ✔ **Tester les topologies Kafka Streams sans cluster Kafka réel**
- ○ Monitorer les métriques en production
- ○ Déployer des topologies en cluster

❓ processing.guarantee=exactly_once_v2 garantit :

- ○ Latence inférieure à 1ms
- ✔ **Traitement sans doublons même en cas de panne — atomique read+write**
- ○ Pas de re-partition
- ○ Performance maximale

03

Kafka Connect

Architecture · Source/Sink · JDBC Connector · SMT · REST API · PostgreSQL

Kafka Connect — Intégration de données sans code

🔌 Apache Kafka Connect

Framework d'intégration de données déclaratif. Connecte Kafka à des systèmes externes (bases de données, fichiers, APIs) sans écrire de code Java. Configuration JSON uniquement. 200+ connecteurs disponibles.

SOURCE Connector

Système externe → Kafka

```
# Exemple : PostgreSQL → Kafka
# Lecture depuis orders table
# Publie dans commerce.orders topic

PostgreSQL → Connector → Kafka Topic

# Cas d'usage :
# CDC avec Debezium (J7)
# JDBC Source (J3 TP)
# FileStream Source (dev/test)
# Elasticsearch Source
# Salesforce Source
```

📌 Ingestion de données externes dans Kafka

SINK Connector

Kafka → Système externe

```
# Exemple : Kafka → Elasticsearch
# Lit commerce.orders topic
# Indexe dans Elasticsearch

Kafka Topic → Connector → Elasticsearch

# Cas d'usage :
# JDBC Sink (PostgreSQL, MySQL)
# Elasticsearch Sink (search)
# S3 Sink (data lake)
# BigQuery Sink (analytics)
# MongoDB Sink (NoSQL)
```

📌 Export de données Kafka vers des systèmes cibles

Architecture Kafka Connect — Workers, Tasks & Converters

🏗️ Architecture interne

Kafka Connect s'exécute en mode Standalone (1 worker) ou Distributed (cluster de workers). Les connecteurs sont distribués en tasks parallèles sur les workers. REST API pour la gestion.

docker-compose.yml — Kafka Connect

```yaml
  connect:
    image: confluentinc/cp-kafka-connect:7.6.0
    ports: ["8083:8083"]
    depends_on: [broker1, broker2, broker3]
    environment:
      CONNECT_BOOTSTRAP_SERVERS: broker1:9092,broker2:9092
      CONNECT_REST_PORT: 8083
      CONNECT_GROUP_ID: "connect-cluster"
      CONNECT_CONFIG_STORAGE_TOPIC: connect-configs
      CONNECT_OFFSET_STORAGE_TOPIC: connect-offsets
      CONNECT_STATUS_STORAGE_TOPIC: connect-status
      CONNECT_CONFIG_STORAGE_REPLICATION_FACTOR: 3
      CONNECT_OFFSET_STORAGE_REPLICATION_FACTOR: 3
      CONNECT_STATUS_STORAGE_REPLICATION_FACTOR: 3
      CONNECT_KEY_CONVERTER: org.apache.kafka.connect.json.JsonConverter
```

Installer des connecteurs

```bash
# Via confluent-hub (dans le container)
confluent-hub install confluentinc/kafka-connect-jdbc:10.7.4
confluent-hub install confluentinc/kafka-connect-elasticsearch:14.0.12
# Redémarrer le worker après installation
```

Composants internes

Worker
Process JVM qui exécute les connecteurs et tasks. Mode standalone ou distributed.

Connector
Plugin qui définit la configuration et crée les tasks.

Task
Unité parallèle de traitement. N tasks = N threads parallèles.

Converter
Transforme les données Kafka ↔ format interne (JSON, Avro, Protobuf).

SMT
Single Message Transform — transformation en vol des messages.

Mode Distributed : plusieurs workers pour la haute disponibilité. Les connecteurs sont automatiquement redistribués si un worker tombe.

JDBC Source Connector — PostgreSQL → Kafka

JDBC Source Connector

Le connecteur JDBC Source lit des tables de base de données et publie les nouvelles lignes dans des topics Kafka. Supporte le polling par timestamp, incrément ou bulk.

Config JDBC Source — Déploiement via REST API

```
# Déployer le connecteur via REST API Kafka Connect
curl -X POST http://localhost:8083/connectors \
  -H "Content-Type: application/json" \
  -d '{
  "name": "jdbc-orders-source",
  "config": {
    "connector.class":
      "io.confluent.connect.jdbc.JdbcSourceConnector",
    "connection.url":
      "jdbc:postgresql://postgres:5432/ecommerce",
    "connection.user": "kafka_user",
    "connection.password": "secret",
    "mode": "timestamp+incrementing",
    "timestamp.column.name": "updated_at",
    "incrementing.column.name": "id",
    "table.whitelist": "orders,customers,products",
    "topic.prefix": "db.",
    "poll.interval.ms": "5000",
    "batch.max.rows": "1000",
    "transforms": "addTopicSuffix",
    "transforms.addTopicSuffix.type":
      "org.apache.kafka.connect.transforms.ReplaceField$Value",
    "numeric.mapping": "best_fit"
```

Modes de lecture JDBC

`bulk`

Relit toute la table à chaque poll. Simple mais coûteux.

`incrementing`

Suit un champ auto-increment. Ne détecte pas les mises à jour.

`timestamp`

Suit un champ timestamp. Détecte INSERT + UPDATE. Risque si clock skew.

`timestamp+incrementing`

Combine les deux. Le plus robuste pour la production.

JDBC Sink Connector — Kafka → PostgreSQL

📤 JDBC Sink Connector

Le connecteur JDBC Sink consomme des topics Kafka et insère/met à jour des lignes dans une base de données. Supporte INSERT, UPSERT (INSERT OR REPLACE) et UPDATE.

Config JDBC Sink — Kafka → PostgreSQL

```
curl -X POST http://localhost:8083/connectors \
  -H "Content-Type: application/json" \
  -d '{
  "name": "jdbc-orders-sink",
  "config": {
    "connector.class":
      "io.confluent.connect.jdbc.JdbcSinkConnector",
    "connection.url":
      "jdbc:postgresql://postgres:5432/warehouse",
    "connection.user": "kafka_sink",
    "connection.password": "secret",
    "topics": "commerce.orders",
    "tasks.max": "3",
    "insert.mode": "upsert",
    "pk.mode": "record_key",
    "pk.fields": "id",
    "auto.create": "true",
    "auto.evolve": "true",
    "batch.size": "1000",
```

Vérifier + gérer les connecteurs

```
curl http://localhost:8083/connectors                          # liste
curl http://localhost:8083/connectors/jdbc-orders-sink/status  # statut
```

Modes insert JDBC Sink

`insert`	INSERT INTO · Pas de gestion des doublons · Rapide
`upsert`	INSERT OR UPDATE · Gère les doublons · Recommandé
`update`	UPDATE uniquement · Erreur si ligne absente

Paramètres import.

`auto.create=true`	Crée la table auto si absente
`auto.evolve=true`	Ajoute les colonnes manquantes
`batch.size=1000`	Batch INSERT pour la perf
`tasks.max=N`	N = nb partitions source topic

auto.create+auto.evolve : pratiques en dev/staging · En prod préférer les migrations DDL explicites (Flyway/Liquibase).

Single Message Transforms (SMT) — Transformation en vol

✂ Single Message Transforms

Les SMT permettent de transformer les messages au sein d'un connecteur Connect, sans écrire de code Java. Chaînables. Appliqués avant l'écriture dans Kafka (source) ou après la lecture (sink).

SMT exemples courants

```
# ─── MASQUER UN CHAMP SENSIBLE (GDPR) ──────────────────
"transforms": "maskCreditCard",
"transforms.maskCreditCard.type":
  "org.apache.kafka.connect.transforms.MaskField$Value",
"transforms.maskCreditCard.fields": "credit_card_number",

# ─── AJOUTER UN TIMESTAMP D'INGESTION ──────────────────
"transforms": "addTimestamp",
"transforms.addTimestamp.type":
  "org.apache.kafka.connect.transforms.InsertField$Value",
"transforms.addTimestamp.timestamp.field": "ingested_at",

# ─── ROUTER VERS DIFFÉRENTS TOPICS SELON CHAMP ─────────
"transforms": "routeByStatus",
"transforms.routeByStatus.type":
  "org.apache.kafka.connect.transforms.TimestampRouter",

# ─── RENOMMER UN CHAMP ─────────────────────────────────
"transforms": "renameId",
"transforms.renameId.type":
  "org.apache.kafka.connect.transforms.ReplaceField$Value",
"transforms.renameId.renames": "orderId:id,custId:customerId",

# ─── FILTRER DES MESSAGES (tombstone records) ──────────
```

SMT natifs disponibles

`MaskField`	Masquer des champs sensibles (GDPR)
`InsertField`	Ajouter un champ (timestamp, topic, partition)
`ReplaceField`	Renommer ou supprimer des champs
`Cast`	Changer le type d'un champ (string→int)
`Filter`	Filtrer/supprimer des messages selon condition
`TimestampConverter`	Convertir les formats de dates
`ValueToKey`	Copier des champs value → key
`ExtractField`	Extraire un champ comme nouvelle value

Les SMT sont limitées à des transformations par message. Pour des transformations complexes → Kafka Streams.

REST API Kafka Connect — Gestion complète

🔧 REST API Kafka Connect

Kafka Connect expose une API REST sur le port 8083 pour créer, lister, monitorer et supprimer des connecteurs. Intégrable avec CI/CD pour déploiement automatisé.

Commandes REST essentielles

```
# ─── CLUSTER ──────────────────────────────────────────────────────
# Version et infos du cluster Connect
curl http://localhost:8083/

# ─── CONNECTEURS ──────────────────────────────────────────────────
# Lister tous les connecteurs
curl http://localhost:8083/connectors

# Créer un connecteur
curl -X POST http://localhost:8083/connectors \
  -H "Content-Type: application/json" \
  -d @connector-config.json

# Voir la config d'un connecteur
curl http://localhost:8083/connectors/my-connector/config

# Mettre à jour la config
curl -X PUT http://localhost:8083/connectors/\
  my-connector/config \
  -H "Content-Type: application/json" \
  -d '{"connector.class": "...", "tasks.max": "4"}'
```

Réponse statut connecteur

```
{
  "name": "jdbc-orders-source",
  "connector": {
    "state": "RUNNING",
    "worker_id": "connect-1:8083"
  },
  "tasks": [
    {
      "id": 0,
      "state": "RUNNING",
      "worker_id": "connect-1:8083"
    },
    {
      "id": 1,
      "state": "RUNNING",
```

Plugins disponibles

curl http://localhost:8083/connector-plugins
→ Liste tous les plugins installés avec leur version et type (SOURCE/SINK)

Écosystème Connecteurs — Les plus utilisés en production

200+ connecteurs disponibles sur Confluent Hub

🗄️ JDBC Source/Sink

`io.confluent.connect.jdbc`

PostgreSQL, MySQL, Oracle, SQL Server, SQLite

kafka-connect-jdbc · Version 10.7+

☁️ S3 Sink

`io.confluent.connect.s3`

Data Lake · Parquet/Avro · Athena/Spark

kafka-connect-s3 · Tiered Storage

🔴 Debezium CDC

`io.debezium.connector.*`

CDC MySQL, PostgreSQL, MongoDB, Oracle, SQL Server

Jour 7 de la formation

🍃 MongoDB Sink/Source

`com.mongodb.kafka.connect`

MongoDB Atlas · Change Streams (Source)

mongodb-kafka-connect-mongodb

🔍 Elasticsearch Sink

`io.confluent.connect.elasticsearch`

Indexation full-text · Kibana dashboards

kafka-connect-elasticsearch

📊 BigQuery Sink

`com.wepay.kafka.connect.bigquery`

Google BigQuery · Analytics · Data warehouse

kafka-connect-bigquery

Confluent Hub : hub.confluent.io · Connectors testés et certifiés · Certains open-source, d'autres sous licence commerciale.

Monitoring Kafka Connect — Métriques et troubleshooting

Métriques JMX et REST

```
# ─── MÉTRIQUES CONNECT ──────────────────────────────────────────

# Via REST : voir toutes les métriques
curl http://localhost:8083/metrics

# Métriques JMX clés (via jconsole ou Prometheus):
# kafka.connect:type=connector-task-metrics,
#   connector=jdbc-orders-source,task=0
#   → batch-size-avg, offset-commit-success-rate
#   → source-record-poll-rate

# ─── LOGS ──────────────────────────────────────────────────────────

docker logs connect -f | grep -E "ERROR|WARN|task"
```

Erreurs courantes et solutions

```
# ERROR: Connection refused → vérifier DB accessible depuis le
container
# FAILED: ClassNotFoundException → plugin non installé
# FAILED: SerializationException → mismatch value.converter
# WARN: offset commit timeout → augmenter offset.flush.timeout.ms
# FAILED: table not found → vérifier table.whitelist et droits SQL
```

Checklist de production

- ☐ 3+ workers pour la haute disponibilité
- ☐ topics config/offset/status RF=3
- ☐ Connecteurs en mode RUNNING
- ☐ Alertes sur connector.state != RUNNING
- ☐ Monitoring source-record-poll-rate
- ☐ consumer.lag sur les topics source
- ☐ backup des configs connecteurs (JSON)
- ☐ error.tolerance=all + DLQ configuré
- ☐ tasks.max ≤ partitions topic source

```
error.tolerance=all
dlq.topic.name=connect.dlq
dlq.topic.replication.factor=3
```

Kafka Connect DLQ : configurer error.tolerance=all + dlq.topic.name pour capturer les erreurs de sérialisation.

Quiz — Section 03 : Kafka Connect

❓ Différence Source vs Sink connector ?

- ○ Source est plus rapide
- ✔ **Source = système externe → Kafka · Sink = Kafka → système externe**
- ○ Sink lit depuis Kafka et écrit dans Kafka
- ○ Source et Sink sont identiques

❓ Qu'est-ce qu'un SMT (Single Message Transform) ?

- ○ Un moniteur de performances
- ✔ **Transformation en vol des messages dans le connecteur sans code Java**
- ○ Un type de connecteur spécial
- ○ Un format de sérialisation

❓ Mode JDBC Source le plus robuste en production ?

- ○ bulk — relit toute la table
- ✔ **timestamp+incrementing — suit timestamp ET auto-increment**
- ○ incrementing uniquement
- ○ timestamp uniquement

❓ Pour gérer les erreurs en production dans Connect :

- ○ Ignorer les erreurs
- ✔ **error.tolerance=all + dlq.topic.name pour capturer les messages en erreur**
- ○ Stopper le connecteur
- ○ Augmenter tasks.max

04

ksqlDB — SQL sur des Streams Kafka

Requêtes persistantes · Tables matérialisées · Streams SQL · Jointures

ksqlDB — Stream processing en SQL

🔍 **ksqlDB**

ksqlDB est un moteur SQL pour Apache Kafka. Il permet de créer des streams et tables sur des topics Kafka et d'exécuter des requêtes SQL continues (push queries) ou ponctuelles (pull queries) — sans écrire de code Java.

Pull Query (ponctuelle)

SELECT état actuel

```
SELECT * FROM orders_table WHERE order_id = 'order-1';
→ Retourne 1 résultat · Comme une requête SQL classique · Pour
l'état courant dans une table matérialisée
```

- ✔ Résultat immédiat unique
- ✔ Equivalent d'un SELECT SQL
- ✔ Nécessite une table matérialisée
- ✔ Faible overhead

Push Query (continue)

SELECT flux en continu

```
SELECT * FROM orders_stream WHERE amount > 100 EMIT CHANGES;
→ Retourne les résultats AU FUR ET À MESURE · Stream infini ·
Pour les nouveaux événements
```

- ✔ Résultats en temps réel
- ✔ Flux continu jusqu'à CTRL+C
- ✔ Idéal pour dashboards live
- ✔ Requête persistante = topic

ksqlDB DDL — Créer Streams et Tables

📐 DDL ksqlDB — CREATE STREAM / TABLE

Un STREAM est backed par un topic Kafka en mode append-only. Une TABLE est backed par un topic Kafka en mode compact (dernier message par clé). Les deux sont persistants après la session.

DDL — Créer Streams et Tables

```
-- Créer un STREAM sur le topic commerce.orders
CREATE STREAM orders_stream (
    order_id    VARCHAR KEY,
    customer_id VARCHAR,
    status      VARCHAR,
    amount      DOUBLE,
    region      VARCHAR,
    created_at  TIMESTAMP
) WITH (
    KAFKA_TOPIC  = 'commerce.orders',
    VALUE_FORMAT = 'JSON',
    PARTITIONS   = 6
);

-- Créer une TABLE sur un topic compact
CREATE TABLE customers_table (
    customer_id VARCHAR PRIMARY KEY,
    email       VARCHAR,
    tier        VARCHAR,  -- 'STANDARD', 'VIP', 'PREMIUM'
    created_at  TIMESTAMP
) WITH (
    KAFKA_TOPIC  = 'commerce.customers',
    VALUE_FORMAT = 'JSON'
);
```

Requêtes essentielles ksqlDB

```
-- Lister les streams et tables
SHOW STREAMS;
SHOW TABLES;
SHOW QUERIES;  -- requêtes persistantes actives

-- Décrire un stream
DESCRIBE orders_stream;
DESCRIBE EXTENDED orders_stream;

-- Lire les messages récents
SELECT *
FROM orders_stream
EMIT CHANGES
LIMIT 10;

-- Agrégation continue
SELECT
    region,
    COUNT(*) AS order_count,
    SUM(amount) AS total_revenue
FROM orders_stream
GROUP BY region
EMIT CHANGES;
```

ksqlDB DML — Agrégations et requêtes persistantes

Agrégations et fenêtres SQL

```
-- CA par heure (tumbling window 1h)
CREATE TABLE hourly_revenue AS
    SELECT
        region,
        WINDOWSTART AS window_start,
        WINDOWEND   AS window_end,
        COUNT(*)    AS order_count,
        SUM(amount) AS total_ca,
        AVG(amount) AS avg_order
    FROM orders_stream
    WINDOW TUMBLING (SIZE 1 HOUR)
    GROUP BY region
    EMIT CHANGES;

-- Top produits (hopping 10min, saut 5min)
```

Jointure Stream-Table en SQL

```
-- Enrichir commandes avec infos client
CREATE STREAM enriched_orders AS
    SELECT
        o.order_id,
        o.amount,
        o.status,
        c.email     AS customer_email,
        c.tier      AS customer_tier,
        CASE
            WHEN c.tier = 'VIP'     THEN o.amount * 0.9
            WHEN c.tier = 'PREMIUM' THEN o.amount * 0.85
            ELSE o.amount
        END AS final_amount
    FROM orders_stream o
    LEFT JOIN customers_table c
```

Pull query sur table matérialisée

```
-- Pull query : résultat immédiat (état courant)
SELECT
    customer_id,
    order_count,
    total_ca
FROM customer_stats
WHERE customer_id = 'cust-123';

-- Windowed pull query
SELECT *
FROM hourly_revenue
WHERE region = 'EU'
```

INSERT INTO et CSAS

```
-- CSAS : Create Stream AS Select (persistent query)
CREATE STREAM vip_orders AS
    SELECT * FROM enriched_orders
    WHERE customer_tier = 'VIP'
    EMIT CHANGES;

-- INSERT INTO : alimenter un stream existant
INSERT INTO orders_stream (
    order_id, customer_id, status, amount
) VALUES ('order-999', 'cust-1', 'CREATED', 299.99);
```

CSAS crée une requête persistante continue (RUNNING) et un topic Kafka en sortie — visible dans SHOW QUERIES.

ksqlDB Setup — Docker Compose et CLI complète

docker-compose.yml — ksqlDB

```
  ksqldb-server:
    image: confluentinc/ksqldb-server:0.29.0
    ports: ["8088:8088"]
    depends_on: [broker1, schema-registry]
    environment:
      KSQL_CONFIG_DIR: /etc/ksqldb
      KSQL_LISTENERS: http://0.0.0.0:8088
      KSQL_BOOTSTRAP_SERVERS: broker1:9092,broker2:9092
      KSQL_KSQL_SCHEMA_REGISTRY_URL: http://schema-registry:8081
      KSQL_KSQL_LOGGING_PROCESSING_STREAM_AUTO_CREATE: "true"
      KSQL_KSQL_LOGGING_PROCESSING_TOPIC_AUTO_CREATE: "true"
      KSQL_KSQL_STREAMS_PROCESSING_GUARANTEE: exactly_once_v2
      KSQL_KSQL_QUERIES_FILE: /etc/ksqldb/queries.sql

  ksqldb-cli:
    image: confluentinc/ksqldb-cli:0.29.0
    depends_on: [ksqldb-server]
    entrypoint: /bin/sh
```

Commandes CLI ksqlDB

```
# Ouvrir la CLI ksqlDB
docker exec -it ksqldb-cli ksql http://ksqldb-server:8088

# Commandes CLI utiles
SET 'auto.offset.reset' = 'earliest';
SHOW STREAMS;    SHOW TABLES;    SHOW QUERIES;
LIST PROPERTIES; -- Voir la configuration ksqlDB
TERMINATE query_id;  -- Arrêter une requête persistante
```

REST API ksqlDB

REST API

```
# Exécuter une requête
curl -X POST http://localhost:8088/query \
  -H "Content-Type: application/vnd.ksql.v1+json"
\
  -d '{
    "ksql": "SELECT * FROM orders_stream EMIT
CHANGES LIMIT 5;",
    "streamsProperties": {
      "ksql.streams.auto.offset.reset":
"earliest"
    }
  }'

# Exécuter DDL
```

Port	8088 (REST) · CLI intégrée
Exactly-Once	exactly_once_v2 disponible
Avro	Intégration Schema Registry native
Headless	Exécuter SQL depuis fichier

Mode headless : KSQL_KSQL_QUERIES_FILE pointe vers un fichier .sql exécuté au démarrage — idéal pour CI/CD.

ksqlDB vs Kafka Streams — Quand utiliser quoi ?

⚖️ Choisir entre ksqlDB et Kafka Streams

ksqlDB est built-on top de Kafka Streams. Pour les cas simples et les prototypes → ksqlDB. Pour les logiques complexes et les intégrations métier → Kafka Streams. Les deux peuvent coexister.

ksqlDB — SQL déclaratif

✔ Idéal pour

- ✔ Requêtes SQL simples et lisibles
- ✔ Prototypage rapide (minutes)
- ✔ Pas de code Java requis
- ✔ Dashboards et monitoring live
- ✔ Transformations simples
- ✔ Analytics exploratoires

✗ Moins adapté pour

Kafka Streams — Java programmatique

✔ Idéal pour

- ✔ Logique métier complexe
- ✔ Tests unitaires complets (TopologyTestDriver)
- ✔ Intégration dans une app Java existante
- ✔ Custom Serdes et Partitioners
- ✔ Gestion fine des erreurs et DLQ
- ✔ Interactive Queries via REST

✗ Moins adapté pour

En pratique : ksqlDB pour les métriques et alertes temps réel · Kafka Streams pour les traitements métier complexes.

TP ksqlDB — Requêtes SQL sur le pipeline e-commerce

🎯 **Objectif : Créer des streams, tables et requêtes persistantes sur commerce.orders**

Script ksqlDB à exécuter

```
-- 1. Ouvrir la CLI
-- docker exec -it ksqldb-cli ksql http://ksqldb-server:8088
-- SET 'auto.offset.reset' = 'earliest';

-- 2. Créer le stream principal
CREATE STREAM orders_stream (
    id VARCHAR KEY, customer_id VARCHAR,
    status VARCHAR, amount DOUBLE, region VARCHAR
) WITH (KAFKA_TOPIC='commerce.orders', VALUE_FORMAT='JSON');

-- 3. Filtrer les commandes haute valeur
CREATE STREAM high_value AS
    SELECT * FROM orders_stream WHERE amount > 200
    EMIT CHANGES;

-- 4. CA par région (fenêtre 1h)
CREATE TABLE regional_revenue AS
    SELECT region,
        SUM(amount) AS total_ca,
        COUNT(*) AS nb_orders
    FROM orders_stream
    WINDOW TUMBLING (SIZE 1 HOUR)
    GROUP BY region EMIT CHANGES;

-- 5. Pull query : voir le CA actuel
```

Résultats attendus

- ☐ SHOW STREAMS → orders_stream visible
- ☐ SHOW STREAMS → high_value visible
- ☐ SHOW TABLES → regional_revenue
- ☐ SHOW QUERIES → 2 requêtes RUNNING
- ☐ Pull query → CA par région
- ☐ Push query → flux en temps réel
- ☐ Produire des msgs → voir flux live
- ☐ BONUS : jointure avec customers_table

BONUS : créer une requête persistante qui enrichit les commandes avec les données clients (LEFT JOIN).

Quiz — Section 04 : ksqlDB

❓ Différence Pull vs Push query ksqlDB ?

- ○ Pull est plus rapide
- ✔ **Pull = résultat immédiat (état courant) · Push = flux continu avec EMIT CHANGES**
- ○ Push renvoie moins de données
- ○ Pas de différence

❓ CREATE STREAM ... AS SELECT crée :

- ○ Un snapshot unique
- ✔ **Une requête persistante RUNNING + un topic Kafka en sortie**
- ○ Une vue SQL temporaire
- ○ Un index de recherche

❓ Quand préférer Kafka Streams à ksqlDB ?

- ○ Toujours — Kafka Streams est plus puissant
- ✔ **Pour des logiques métier complexes avec tests unitaires et intégration Java**
- ○ Pour les requêtes simples
- ○ Pour les dashboards live

❓ WINDOW TUMBLING (SIZE 1 HOUR) signifie :

- ○ Fenêtres de 1h qui se chevauchent
- ✔ **Fenêtres disjointes de 1h — chaque message dans 1 seule fenêtre**
- ○ Fenêtres glissantes
- ○ Sessions d'1h

05

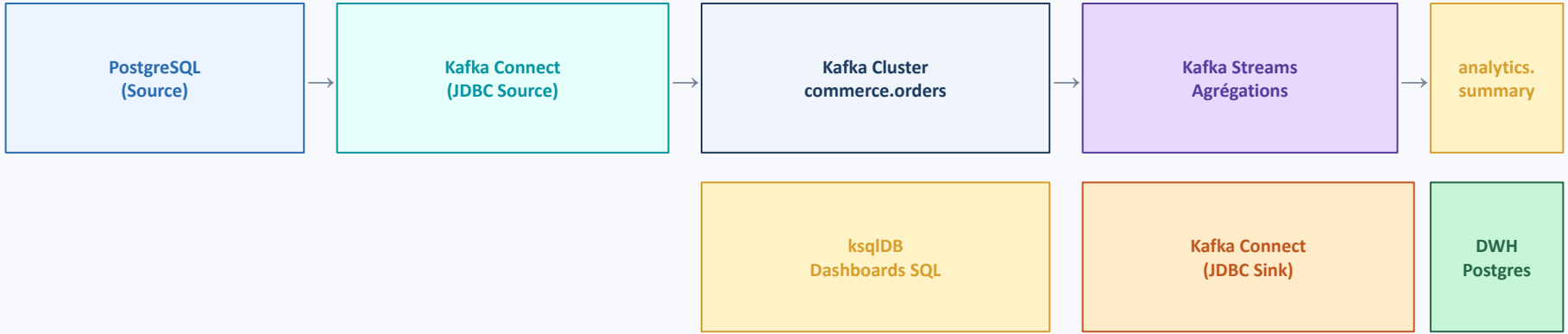
TP — Pipeline temps réel bout en bout

Kafka Streams Java · ksqlDB · JDBC Source · JDBC Sink · PostgreSQL

Architecture TP Jour 3 — Pipeline temps réel complet

🏗️ Pipeline E-commerce Jour 3

Extension du pipeline J2 : ajout de Kafka Connect (JDBC Source depuis PostgreSQL), Kafka Streams (agrégation temps réel), ksqlDB (dashboards SQL) et JDBC Sink (résultats vers Data Warehouse).



Topics Kafka du TP Jour 3

`db.orders (JDBC Source)`

Auto-créé par Connect depuis PostgreSQL orders table

`commerce.orders.streams`

KStream traité par Kafka Streams (enrichi + filtré)

`analytics.hourly-summary`

Agrégats horaires produits par Kafka Streams

`analytics.regional-stats`

CA par région produit par ksqlDB requête persistante

EX 1 — JDBC Source : PostgreSQL → Kafka (25 min)

🎯 **Objectif : Connecter PostgreSQL à Kafka via JDBC Source Connector**

Setup PostgreSQL + Connecteur

```
-- 1. PostgreSQL : créer la table orders
CREATE TABLE orders (
    id          SERIAL PRIMARY KEY,
    order_id    VARCHAR(50) UNIQUE NOT NULL,
    customer_id VARCHAR(50),
    status      VARCHAR(20),
    amount      DECIMAL(10,2),
    region      VARCHAR(20),
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);
-- Insérer quelques lignes de test
INSERT INTO orders (order_id, customer_id, status, amount, region)
VALUES ('ord-001', 'cust-1', 'CREATED', 99.99, 'EU'),
       ('ord-002', 'cust-2', 'PAID',   249.00, 'US');

-- 2. Déployer le connecteur JDBC Source
curl -X POST http://localhost:8083/connectors \
  -H "Content-Type: application/json" \
  -d '{
    "name": "jdbc-orders-source",
    "config": {
      "connector.class": "io.confluent.connect.jdbc.JdbcSourceConnector",
      "connection.url": "jdbc:postgresql://postgres:5432/ecommerce",
      "connection.user": "postgres",
```

Validation de l'exercice

- ☐ PostgreSQL accessible : docker exec -it postgres psql
- ☐ Connecteur créé : curl /connectors
- ☐ Statut RUNNING : /connectors/status
- ☐ Topic créé : kafka-topics.sh --list
- ☐ Messages dans db.orders : Kafka UI
- ☐ Insérer 10 nouvelles lignes en SQL
- ☐ Observer l'ingestion dans Kafka UI
- ☐ Vérifier le polling interval (3s)

Clé : observer le LAG entre une insertion PostgreSQL et l'apparition dans Kafka (< poll.interval.ms = 3s).

EX 2 — Kafka Streams : Agrégation temps réel (35 min)

🎯 **Objectif : Implémenter une topologie Kafka Streams qui agrège le CA par région par heure**

OrderAggregator.java — Squelette

```
@Component
public class OrderAggregator {

    public Topology buildTopology(StreamsBuilder builder) {

        // TODO 1 : Créer le KStream depuis db.orders
        KStream<String, String> rawOrders =
            builder.stream("db.orders");

        // TODO 2 : Désérialiser en objets Order
        KStream<String, Order> orders = rawOrders
            .mapValues(json -> /* parser JSON */);

        // TODO 3 : Grouper par région
        KGroupedStream<String, Order> byRegion =
            orders.groupBy(
                (key, order) -> /* ??? */);

        // TODO 4 : Fenêtre tumbling 1h + agrégation CA
        KTable<Windowed<String>, Double> hourlyRevenue =
            byRegion.windowedBy(
                TimeWindows.ofSizeWithNoGrace(
                    Duration.ofHours(1)))
            .aggregate(
                () -> 0.0,
```

Checklist & BONUS

- ☐ Application démarre sans erreur
- ☐ Topic analytics.hourly-summary créé
- ☐ Messages agrégés visibles (Kafka UI)
- ☐ CA correct par région (comparer source)
- ☐ TopologyTestDriver test unitaire OK
- ☐ Lancer 2 instances → rebalancing
- ☐ BONUS : ajouter count() par région
- ☐ BONUS : Interactive Query REST /revenue/{region}

Test unitaire : TopologyTestDriver avec pipeInput de 10 commandes EU · Assert total CA = somme attendue.

EX 3 — ksqlDB : Dashboard SQL temps réel (20 min)

🎯 **Objectif : Créer des requêtes ksqlDB pour le dashboard e-commerce en temps réel**

Script ksqlDB complet

```
-- Ouvrir la CLI
-- docker exec -it ksqldb-cli ksql http://ksqldb-server:8088
-- SET 'auto.offset.reset' = 'earliest';

-- 1. Créer le stream sur db.orders (JDBC Source)
CREATE STREAM orders_raw (
    order_id VARCHAR KEY, customer_id VARCHAR,
    status VARCHAR, amount DOUBLE, region VARCHAR,
    updated_at BIGINT
) WITH (
    KAFKA_TOPIC  = 'db.orders',
    VALUE_FORMAT = 'JSON'
);

-- 2. CA et nombre de commandes par région (rolling 1h)
CREATE TABLE regional_dashboard AS
    SELECT
        region,
        COUNT(*)    AS nb_orders,
        SUM(amount) AS total_ca,
        AVG(amount) AS avg_amount,
        MAX(amount) AS max_order
    FROM orders_raw
    WINDOW TUMBLING (SIZE 1 HOUR)
    GROUP BY region EMIT CHANGES;
```

Résultats attendus

Exemple output ksqlDB

```
-- Pull query regional_dashboard :
+---------+-----------+-----------+
| REGION  | NB_ORDERS | TOTAL_CA  |
+---------+-----------+-----------+
| EU      | 45        | 4532.50   |
| US      | 38        | 3890.00   |
| ASIA    | 22        | 1890.99   |
+---------+-----------+-----------+

-- Push query vip_orders :
| ord-999 | 999.99 | EU |
(nouvelle ligne à chaque >500€)
```

- ☐ SHOW STREAMS → 3 streams actifs
- ☐ SHOW TABLES → regional_dashboard
- ☐ Pull query → CA par région correct
- ☐ Push query → flux VIP en direct
- ☐ INSERT → apparaît dans push query
- ☐ SHOW QUERIES → 2 requêtes RUNNING

Le dashboard ksqlDB répond en < 10ms (pull query) grâce à la table matérialisée — pas de scan de topic.

EX 4 — JDBC Sink : Kafka → Data Warehouse (20 min)

🎯 **Objectif : Persister les agrégats Kafka Streams dans PostgreSQL via JDBC Sink**

Setup JDBC Sink Connector

```
-- 1. Créer la table de destination (Data Warehouse)
-- Sur la base PostgreSQL warehouse :
CREATE TABLE hourly_revenue_dw (
    region       VARCHAR(20),
    window_start TIMESTAMP,
    window_end   TIMESTAMP,
    total_ca     DECIMAL(15,2),
    nb_orders    INTEGER,
    avg_amount   DECIMAL(10,2),
    PRIMARY KEY (region, window_start)
);

-- 2. Déployer le JDBC Sink
curl -X POST http://localhost:8083/connectors \
  -H "Content-Type: application/json" \
  -d '{
    "name": "jdbc-analytics-sink",
    "config": {
      "connector.class":
        "io.confluent.connect.jdbc.JdbcSinkConnector",
      "connection.url":
        "jdbc:postgresql://postgres:5432/warehouse",
      "connection.user": "postgres",
      "connection.password": "postgres",
      "topics": "analytics.hourly-summary",
```

Pipeline complet vérifié

Flow de bout en bout

```
PostgreSQL (source)
    ↓ JDBC Source (Connect)
Kafka: db.orders
    ↓ Kafka Streams
Kafka: analytics.hourly-summary
    ↓ JDBC Sink (Connect)
PostgreSQL (warehouse)

# Vérification : insérer une commande
# dans PostgreSQL source et vérifier
# qu'elle apparaît dans warehouse en < 10s !
```

- ☐ Sink connecteur RUNNING
- ☐ Table hourly_revenue_dw créée
- ☐ Données visibles en SQL
- ☐ Pipeline E2E < 10 secondes

EX 5 — Test E2E et résilience du pipeline (20 min)

🎯 **Objectif : Valider le pipeline complet et tester la résilience**

Tests de résilience

```
# ─── TEST 1 : Latence end-to-end ──────────────────────────
# Insérer une commande dans PostgreSQL source
INSERT INTO orders (order_id, customer_id, status, amount, region, updated_at)
VALUES ('perf-001','cust-1','CREATED',199.99,'EU', NOW());
# Mesurer : temps jusqu'à apparition dans hourly_revenue_dw
# Attendu : < 10 secondes (poll 3s + Streams + Sink)

# ─── TEST 2 : Résilience JDBC Source ──────────────────────
docker stop postgres  # Simuler panne DB source
# Observer : Connect passe en FAILED
# Observer : Kafka Streams continue (messages en attente)
docker start postgres
curl -X POST http://localhost:8083/connectors/\
  jdbc-orders-source/restart  # Relancer le connecteur
# Observer : reprise depuis le dernier offset commité

# ─── TEST 3 : Résilience Kafka Streams ────────────────────
# Tuer l'instance Kafka Streams
kill -9 $(pgrep -f OrderAggregator)
# Observer : messages s'accumulent dans db.orders
# Redémarrer l'application Java
# Observer : reprise et rattrapage du lag

# ─── TEST 4 : Charge ──────────────────────────────────────
# Générer 1000 commandes en batch SQL
```

Résultats attendus

Latence E2E

→ < 10 secondes

Résilience Source

→ Reprise auto après restart

Résilience Streams

→ Rattrapage lag automatique

1000 commandes

→ Traitées en < 30 secondes

DWH

→ Données cohérentes en SQL

✔ **Pipeline industriel validé !**

Patterns d'architecture — Kafka comme hub central

Patterns d'intégration avec Kafka en production

🔄 Lambda Architecture

Traitement batch (Spark) + streaming (Kafka Streams) en parallèle · Résultats fusionnés · Robustesse maximale · Complexité opérationnelle élevée

💡 Avantage : tolérance aux pannes streaming · Inconvénient : duplication de logique

🗄️ Event Sourcing + CQRS

Topics Kafka = source de vérité · KTable = read model · Commands → Events → Projections · Audit log natif · Microservices découplés

💡 Avantage : traçabilité complète · Inconvénient : complexité architecturale

⚡ Kappa Architecture

Kafka Streams uniquement · Replay depuis le beginning pour le batch · Plus simple que Lambda · Requiert Kafka comme source de vérité

💡 Avantage : 1 seule codebase · Inconvénient : replay coûteux si historique long

📊 Data Mesh avec Kafka

Chaque domaine publie ses events · Schema Registry = contrat · Topics = produits data · Connect = intégrations · Gouvernance décentralisée

💡 Avantage : autonomie des équipes · Inconvénient : gouvernance complexe

Kappa Architecture = standard moderne · Kappa + Kafka Streams = stack choisi par la plupart des scale-ups.

Debugging Jour 3 — Kafka Streams, Connect & ksqlDB

Kafka Streams — Debug

```
# Voir l'état de l'application Streams
# Via health endpoint Spring Boot
curl http://localhost:8080/actuator/health

# Métriques détaillées JMX
# Bean: kafka.streams:type=stream-metrics
# Métriques: process-rate, commit-rate, lag

# Visualiser la topologie
System.out.println(topology.describe());
# → Copier sur zz85.github.io/kafka-streams-viz

# Logs Kafka Streams
log4j.logger.org.apache.kafka.streams=DEBUG
# → Voir chaque message traité (attention: verbose)
```

Kafka Connect — Debug

```
# Voir les erreurs de tâches
curl http://localhost:8083/connectors/\
  my-connector/status | python3 -m json.tool

# Topic de logs Connect
kafka-console-consumer.sh \
  --topic _connect-status

# Activer les logs DEBUG pour un connector
curl -X PUT http://localhost:8083/admin/loggers/\
  io.confluent.connect.jdbc \
  -H "Content-Type: application/json" \
  -d '{"level":"DEBUG"}'

# Inspecter les offsets Connect
```

ksqlDB — Debug

```
# Vérifier les erreurs de requêtes
SHOW QUERIES EXTENDED;  -- Détails par requête

# Expliquer le plan d'exécution
EXPLAIN ANALYZE SELECT * FROM orders_stream
    WHERE amount > 100 EMIT CHANGES;

# Topic de logging ksqlDB
kafka-console-consumer.sh \
  --topic _confluent-ksql-default__command_topic
# → Voir les commandes DDL exécutées
```

Commandes de diagnostic global

```
# Tout voir en une commande
for topic in _connect-configs _connect-offsets \
  _connect-status; do
  echo "=== $topic ==="
  kafka-topics.sh --bootstrap-server localhost:19092 \
    --describe --topic $topic
done

# Vérifier tous les consumer groups actifs
kafka-consumer-groups.sh \
  --bootstrap-server localhost:19092 \
  --list | grep -E "streams|connect|ksql"
```

Kafka Streams + Connect + ksqlDB créent chacun des consumer groups internes — surveiller leur LAG.

Récapitulatif Jour 3 — 6 compétences acquises + Aperçu J4

🌊 Kafka Streams

KStream/KTable · topologie · stateless transforms · state stores RocksDB · exactly_once_v2

🔍 ksqlDB

CREATE STREAM/TABLE · Push/Pull queries · Fenêtres SQL · Requêtes persistantes

📊 Agrégations & Fenêtres

count · aggregate · Tumbling/Hopping/Session · jointures S-T · GlobalKTable

🏗️ Pipeline E2E

PostgreSQL → Connect → Kafka → Streams → Connect → DWH · Latence < 10s

🔌 Kafka Connect

Source/Sink · JDBC Connector · SMT · REST API · PostgreSQL · 200+ connecteurs

🧪 Testing

TopologyTestDriver · Tests unitaires sans Kafka · Debugging J3 complet